Maintaining Draft Policies Under Meta Drift: What Retraining and Refresh Are Worth

Anonymous Authors

***Abstract*—Learned draft assistants deploy into games whose balance keeps changing. Using 1.94 million ranked Heroes of the Storm drafts spanning 45 game builds over 4.5 years, we measure what that drift costs a deployed draft policy and what each kind of maintenance buys back. In a replayed 2.8-year deployment, a model that is never maintained loses 1.02 percentage points of accuracy against one retrained after every build. Retraining every three builds recovers 0.99 of those points; recomputing the model's aggregate statistics at every build, with the weights fixed, recovers 0.36. Once retraining is on a schedule, refreshing statistics between retrains adds only 0.03 to 0.07 points. Accuracy understates policy differences: the most accurate of eight training regimes, 0.12 points above our maintained model, drafts structurally broken teams at 24% against 13%. Head-to-head, scored on 293,000 games from five later builds that no agent's statistics include, staleness costs in proportion to its length: an agent trained at the 2026 cutoff leads agents trained one and two years earlier by 0.56 and 1.39 percentage points of realized future-meta hero win rate, the one- and two-year agents differ from each other ($p = 0.03$), and a maintained agent leads the two-year agent by 1.77 points, 97% of which survives matching its training volume. Within one cutoff the maintained agent leads by 0.58 points ($p = 0.07$). The two-year agent counters newer agents' picks better than they counter its own, significantly in two of four matchups. An outcome-shift detector fires on 16 of 21 build boundaries with documented changes and on 1 of 6 without (Fisher $p = 0.015$), yet as a refresh trigger it does no better than a calendar. Learned judges drift too: judges trained through early 2022 reverse the realized verdict of a 2026 contest, and judges from 2025 on agree with it.**[1]

***Index Terms*—Concept drift, dataset shift, MOBA drafting, model maintenance, non-stationarity.**

I. Introduction

In prior work we built a draft assistant for Heroes of the Storm and deployed it in a browser where anyone can draft against it [1]. That paper froze the world on purpose: every experiment ran on a snapshot of 1.95 million ranked drafts, pinned in May 2026, so that results would be reproducible. The game did not hold still. Across the window our corpus covers, the developer shipped 45 game builds, and the balance changes among them moved the meta under every model we trained.

This paper restores the time axis and asks what a deployed draft policy loses when nobody maintains it, and what each maintenance action buys back. The two actions we price are retraining the model and refreshing the aggregate statistics its features are computed from (hero, pair, map, and composition win rates), with the weights left alone. Refresh is nearly free; retraining costs a training run. Practitioners would like to know whether the cheap action is enough.

We answer in four parts. First, a replayed 2.8-year deployment prices both actions at the accuracy level. Never maintaining costs 1.02 percentage points (pp) of accuracy relative to retraining after every build. Retraining every three builds recovers 0.99pp of that, and refreshing statistics without retraining recovers 0.36pp. With retraining on a schedule, refresh between retrains adds 0.03 to 0.07pp, and every cadence from one to six builds lands within 0.08pp of retraining constantly. Second, accuracy understates what drift does to policies. Among eight training regimes, the patch-embedding regime has the best future accuracy, 0.12pp above our maintained model, yet drafting greedily it assembles structurally broken teams nearly twice as often (24% against 13%). Third, MCTS agents trained at different cutoffs draft against each other and are scored against realized outcomes of games that no agent's statistics include. Staleness costs in proportion to its length: an unmaintained agent trained at the 2026 cutoff leads agents built with the same recipe one and two years earlier by +0.56pp and +1.39pp of realized future-meta hero win rate, the one-year agent leads the two-year agent by +0.70pp ($p = 0.03$), and the maintained agent leads the two-year agent by +1.77pp. Matching the maintained agent's training volume to the two-year agent's keeps 97% of that gap. Within one cutoff, the maintained agent leads by +0.58pp ($p = 0.067$), positive on every truth set but not significant at six agents per side. Pairwise play ages differently: the two-year agent counters newer agents better than they counter it, significantly in two of four matchups. Fourth, the machinery a maintainer needs around these decisions: a detector of balance changes from outcomes alone, measured against the developer's patch notes, and the observation that learned judges age like the models they judge.

Scoping notes. Each learned judge we tried scores this contest by its own era (Section VIII), so the head-to-head results are scored on realized future outcomes. And the retrospective design can be tuned to the future it holds out, so we registered forward tests on OSF before submission (Section XI). Throughout, statistics used as training features are computed causally or out of fold, because statistics that include a row's own outcome leak its label into its features (Section II).

II. Setting, Corpus, and Names

We use the pinned corpus of our earlier work: 1,943,338 ranked Heroes of the Storm drafts on major patch 2.55,

[1] https://anonymous.4open.science/status/moba-draft-policies

December 2021 through May 2026, a window over which the hero pool is fixed at 90. What is new is the time structure. The corpus spans **45 game builds**, of which 28 have at least 20,000 games ("sizable"). The balance unit is the build: hero tuning ships per build, and one marketing-level patch can span many builds. Build counts recur in this paper, nested as follows. Of the 45 builds, 28 are sizable. Forty builds map to scripted patch notes, and five have no published notes and are treated as no-change builds (Section VII). Consecutive sizable builds define 27 boundaries that are detectable in principle; the notes for 21 of them mention at least one hero and the notes for 6 mention none. Skill is bucketed into three tiers: low (Bronze and Silver), mid (Gold and Platinum), and high (Diamond and Master); the 5,749 games without a league tier are excluded. Replacing the tier one-hot with continuous average match rating changes future accuracy by -0.008pp, and appending it changes it by +0.021pp, both inside seed noise.

All value functions share the architecture of our earlier work: a 283-input MLP over hero identities, map, tier, and aggregate-statistic features. The aggregate statistics come from our own corpus, per build and cumulatively to each build. The temporal split is a hard cutoff at build 2.55.14.95918 (February 2026). Training uses builds up to the cutoff; the *future window* is the seven strictly later builds, about 144,000 games. Where statistics come from matters in this paper, so we state it per arm. The MAINTAINED recipe gives every row statistics cumulative through the build before its own, which is causal by construction. The cutoff-frozen recipes give deployment rows the cumulative statistics at the cutoff, and their training rows the same statistics computed out of fold, as described next. Accuracies are labeled either *future-window* (the seven builds) or *deployment-weighted* (a 2.8-year replay, Section IV); the two windows give different numbers for the same policy.

Out-of-fold statistics. Giving every training row the statistics frozen at the cutoff leaks labels: a December 2021 game's hero, pair, map, and composition win rates would then be computed from every game through February 2026, including that game, a known hazard with target statistics [8], [9]. Each replay therefore receives one of five hash folds, and a training row gets the cutoff statistics with its own fold's games removed; deployment rows keep the unmodified cutoff statistics, which contain no deployment game. Built from the full counts, the same code reproduces the stored cutoff statistics to the last digit. The leak matters. For all-history training, moving from leaky to out-of-fold statistics lowers in-period validation accuracy from 59.04% to 57.31%, the calibration slope on the future window (the slope of a logistic recalibration of the model's predictions; 1 is calibrated) rises from 0.70 to 0.96 (MAINTAINED: 0.92), and the rate of structurally broken teams under greedy drafting falls from 41.8% to 20.7%. In-period validation cannot reveal the leak, because validation rows carry it too; the leaky models look calibrated there (slope 1.0). Because training rows see statistics from four fifths of the games and deployment rows from all of them, cells near the count thresholds could go missing more often in training; on a 60,000-game sample the lookup rates differ by at most 0.007 percentage points. We report the leaky versions only as labeled controls, next to the leak-free results they replace.

TABLE I
ARM NAMES. "CUTOFF" IS BUILD 2.55.14.95918 (FEBRUARY 2026).

Name	Meaning
MAINTAINED	value function trained with each row's statistics cumulative through the build before its own; its MCTS policy trained with kernel statistics through the last completed build (May 2026)
MAINTAINED-D90	as MAINTAINED, with statistics decayed on a 90-day half-life and counts shrunk toward the cumulative value (100 pseudo-games) (Sec. IV)
MAINTAINED-AT-CUTOFF	the MAINTAINED value functions, with the MCTS policy trained on kernel statistics frozen at the cutoff
UNMAINTAINED	value function trained on cutoff-frozen statistics with out-of-fold training rows; policy trained on cutoff statistics
STALE-1YR / STALE-2YR	the UNMAINTAINED recipe with the cutoff one and two years earlier, and an opponent model of that era
UNMAINTAINED-LEAKY	UNMAINTAINED trained on leaky statistics, whose training rows saw their own outcomes; the registered head-to-head's control (Sec. XI)
frozen-statistics recipe	in the replay (Sec. IV), a value function trained at the replay's first cutoff (July 2023) on out-of-fold cutoff statistics and deployed with those statistics frozen
LEAKY-FROZEN	the earlier paper's model, whose statistics saw the future; a control

The agent under maintenance has three learned parts. A win-probability model (the MLP above) scores completed drafts. An opponent model, trained by behavioral cloning on human next-picks, supplies realistic adversaries. A policy network, trained by AlphaZero-style Monte Carlo tree search (MCTS) self-play against the opponent model with the win-probability model at the leaves, proposes picks and bans; the search kernel computes the win-probability features from a table of aggregate statistics. Evaluation uses two instruments. Head-to-head drafting: two agents alternate through the real 16-step ban-pick protocol, each acting greedily from its policy head, and the terminal drafts are scored against statistics of games actually played later (Section V). Structural metrics: the degenerate-composition rate counts teams missing a role that role-level statistics show is required (a healer or a frontline), and synergy and counter scores measure pairwise interactions beyond hero strength. These instruments come from our earlier work [1].

Because "frozen" can mean several things here, we name the arms once (Table I) and never use the word bare.

III. ACCURACY UNDERSTATES POLICY DAMAGE

Table II compares eight training regimes under identical architecture and budget: all-history training on cutoff-frozen statistics, recency windows of 3, 6, and 12 builds, exponential sample decay with 90- and 365-day half-lives, a learned patch

TABLE II

TRAINING REGIMES, LEAK-FREE, WITH LEAKY STATISTICS FOR COMPARISON (3 SEEDS EACH). VAL: IN-PERIOD VALIDATION ACCURACY; FUTURE: FUTURE-WINDOW ACCURACY; SLOPE: CALIBRATION SLOPE ON THE FUTURE WINDOW; DEGEN: BROKEN COMPOSITIONS UNDER GREEDY DRAFTING AGAINST AN OPPONENT MODEL TRAINED TO THE CUTOFF (3 SEEDS × 500 DRAFTS). MAINTAINED HAS NO LEAKY VERSION.

	leak-free (out-of-fold)				leaky		
Regime	Val	Future	Slope	Degen %	Val	Future	Degen %
all history	57.3	56.68	0.96	20.7	59.0	56.34	41.8
window 3	57.3	56.59	0.97	42.5	59.5	56.56	48.8
window 6	57.9	56.73	0.90	40.2	59.8	56.61	48.0
window 12	56.5	56.71	0.92	30.1	59.0	56.52	46.2
decay 90d	56.8	56.79	1.07	48.3	58.7	56.64	49.2
decay 365d	57.2	56.87	0.98	30.1	59.0	56.47	44.5
patch embed	57.5	**57.10**	0.98	24.0	59.2	56.73	42.5
MAINTAINED	57.0	56.98	0.92	**12.7**			

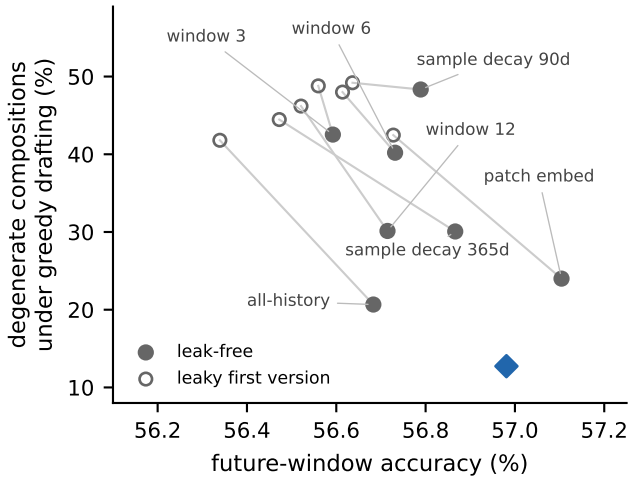


Fig. 1. Future-window accuracy against greedy degenerate-composition rate for the eight training regimes, leak-free (filled) and in the with leaky statistics (hollow, joined to their leak-free counterpart). Removing the leak moves the all-history regimes down and to the right; the short-history regimes stay degenerate.

embedding, and MAINTAINED, which trains on all history with statistics cumulative through each row's previous build.

Leak-free, the regimes' future-window accuracies span 56.59% to 57.10%, 0.5pp. Their rates of broken teams under greedy drafting span 12.7% to 48.3%, a factor of nearly four. Accuracy says little about which regime is which. Across the seven cutoff-frozen regimes, the correlation between future accuracy and degeneracy is -0.38 ($n = 7$, not significant). The patch-embedding regime is the most accurate of all eight, 0.12pp above MAINTAINED, and drafts broken teams at 24.0% against 12.7% (Welch $t = 6.9$, 3.7 df, $p = 0.003$, three seeds each). Selecting among these regimes by accuracy would pick the one that drafts broken teams twice as often.

History length accounts for most of the spread: regimes that see mostly recent builds, by a 3- or 6-build window or by 90-day sample decay, draft broken teams 40% to 48% of the time, and regimes that keep all or most history sit at 21% to 30%. Statistics aligned with each game's own era lower the rate further: all-history training on cutoff-frozen statistics drafts broken teams 20.7% of the time against 12.7% for MAINTAINED (Welch $t = 3.7$, 3.0 df, $p = 0.04$). Three seeds per arm make these tests weak, and we do not detect the difference at the agent level (Section V).

Leaky statistics distort both measures (Figure 1). With each row's own outcome in its statistics, the all-history regimes draft broken teams far more often (41.8% against 20.7%, 44.5% against 30.1%, and 42.5% against 24.0%; Welch $t = 8.9$, 3.6 df, $p = 0.001$ for all history), the short-history regimes reach 46% to 49%, every regime's future accuracy falls by 0.03 to 0.4pp, and the correlation between accuracy and degeneracy within the standard regimes turns positive ($+0.36$). What remains is that at equal or better accuracy, a policy can draft a broken team twice as often, so policy metrics have to be measured directly.

TABLE III

DEPLOYMENT REPLAY OVER 2.8 YEARS: DEPLOYMENT-WEIGHTED REGRET AGAINST RETRAINING AFTER EVERY BUILD, ONE TRAINING SEED. "REFRESH" RECOMPUTES STATISTICS AT EVERY BUILD; "NO REFRESH" RECOMPUTES THEM ONLY AT RETRAINS. [a]THREE SEEDS.

Policy	Retrain	Refresh	Regret (pp)
reference: retrain every build	35	35	0.000
never retrain, never refresh	0	0	+1.021
frozen-statistics recipe, leak-free[a]	0	0	+1.01 to 1.35
frozen-statistics recipe, leaky	0	0	+1.698
never retrain, refresh every build	0	35	+0.662
never retrain, refresh every 3rd	0	11	+0.685
never retrain, 14 even refreshes	0	14	+0.675
never retrain, detector refreshes	0	14	+0.677
retrain every 3, no refresh	11	11	+0.028
retrain every 3, refresh	11	35	−0.002
retrain every 6, no refresh	5	5	+0.146
retrain every 6, refresh	5	35	+0.081
finetune every 3, refresh	11	35	−0.005
retrain every 9 / 12, refresh	3 / 2	35	+0.17 / 0.20

IV. WHAT RETRAINING AND REFRESH ARE WORTH

We replay a 2.8-year deployment forward across 36 builds, starting from about 640,000 games of history at build 2.55.3.89754 (July 2023), and score every maintenance policy by games-weighted accuracy and by regret against retraining after every build (Table III). Every maintenance-policy row trains on causal statistics with seed 42; the frozen-statistics rows are the out-of-fold recipe (three seeds) and the leaky construction (one seed). Differences between rows of a few hundredths of a point are within the noise one training seed adds.

Doing nothing costs 1.02pp over the deployment. Refreshing the statistics every build, with the weights fixed, recovers 0.36pp of it, 35%. Retraining every three builds recovers 0.99pp even when the statistics are recomputed only at each retrain, and refreshing between retrains then adds 0.03pp; at a six-build cadence it adds 0.07pp. Retraining does the repair. Refresh recovers a third of the loss when retraining is impossible, and adds a few hundredths of a point once retraining is scheduled. With refresh, cadences from one to six builds land within 0.08pp of the reference, and the regret grows past that (0.17pp at nine builds, 0.20pp at twelve). A

warm-start finetune on the last six builds matches a full retrain. In calendar terms the replay's builds average 29 days apart, so a three-to-six-build cadence is quarterly to twice a year.

Refresh does little on the future window of the main split too. Holding the MAINTAINED checkpoints fixed and freezing their statistics at the cutoff lowers future-window accuracy by only 0.035pp (56.98% to 56.95%), a three-month horizon. The leak-free all-history model scores 56.68%. So at the 2026 cutoff, 0.27 of the 0.30pp that separates the two training recipes comes from training against statistics aligned with each game's own era, and 0.035pp from refresh. In the replay the recipe difference is within seed noise: the frozen-statistics recipe's three seeds lose 1.01 to 1.35pp, bracketing the never-refreshed causal model's 1.02pp. The leaky construction puts the frozen-statistics recipe at 1.70pp.

Decayed aggregates. If recomputing statistics helps a little, old games may deserve less weight inside them. Decaying the aggregates exponentially, with the decay applied per build at each build's last game date, raises future-window accuracy from 56.98% (cumulative) to 57.17% with a 90-day half-life, 57.23% with a 90-day half-life and counts shrunk toward the cumulative value (100 pseudo-games), and 57.18% with a 365-day half-life. Paired on the same rows, with each arm's three seeds averaged and standard errors clustered by game, the gains are +0.19pp ($z = 3.0$), +0.24pp ($z = 4.2$), and +0.20pp ($z = 4.2$). The half-life itself is not identified. A nested check that repeats the selection at an earlier cutoff and scores the four builds before the 2026 cutoff finds no reliable difference from cumulative statistics (-0.02, +0.03, and +0.06pp; $|z| \leq 1.3$). We therefore recommend decaying the aggregates without claiming a best half-life. Decaying each signal class at its own measured rate does no better: hero-family statistics on a 90-day clock with pair and composition statistics on a 365-day clock score 57.07%, the reverse assignment 57.14%.

What refresh touches. A partial-refresh ablation on the fixed MAINTAINED checkpoints swaps statistics by signal class at test time. Every effect is within ± 0.07pp over the three-month horizon. Refreshing hero-level statistics alone lowers accuracy by 0.02pp; refreshing pair and composition statistics alone raises it by 0.07pp, more than refreshing everything (+0.035pp). Raw pair win rates drift as fast as hero win rates, and the pair features are normalized by hero win rates, so the pair-only cell also carries current hero strength, and the hero-only cell feeds the model mismatched numerators and denominators; we draw no conclusion about which signal class matters.

V. The Cost of Neglect, Head-to-Head

Accuracy comparisons leave open whether any of this matters when policies play. We answer with direct competition. Two agents draft against each other in every pairing of their seeds, both first-pick orderings, 40 drafts per ordering over stratified maps and tiers, each agent acting greedily from its policy head. At play time no agent uses aggregate statistics; statistics enter only through the value function and the kernel's statistics table during MCTS training. "Maintenance" at the agent level therefore means a policy trained against refreshed statistics, which is a retrain.

TABLE IV

Head-to-head results on out-of-sample builds. Δ: the first agent's advantage in future-meta hero win rate (pp, crossed seed random effects, t reference), on all five builds and on the four 2.55.17 builds alone. Net: team win-probability gap from a realized-outcome index with hero, synergy, counter, and role-composition terms and three structure indicators (no healer, no frontline, stacked role), weights fitted on held-out real games (two cross-fitting folds), on the same 292,902 games. M: maintained; M-d90: maintained-d90; M-cut: maintained-at-cutoff; U: unmaintained; S1, S2: stale-1yr, stale-2yr; M-vol: maintained trained on S2's row count.

Matchup	Seeds	Δ (pp)	t (df)	2.55.17	Net
staleness					
U vs S1	6×3	$+0.56 \pm 0.19$	2.9 (8)	$+0.79$	+2.1
U vs S2	6×6	$+1.39 \pm 0.26$	5.4 (10)	$+1.50$	+4.7
S1 vs S2	3×6	$+0.70 \pm 0.26$	2.7 (7)	$+0.66$	+2.5
M vs S1	6×3	$+1.02 \pm 0.20$	5.1 (7)	$+1.16$	+4.7
M vs S2	6×6	$+1.77 \pm 0.26$	6.9 (10)	$+1.82$	+7.7
M-vol vs S2	3×6	$+1.71 \pm 0.23$	7.3 (7)	$+1.76$	+7.5
same cutoff					
M vs U	6×6	$+0.58 \pm 0.28$	2.0 (10)	$+0.51$	+3.2
M vs M-cut	6×3	$+0.21 \pm 0.25$	0.8 (8)	$+0.14$	+1.5
M-cut vs U	3×6	$+0.39 \pm 0.28$	1.4 (7)	$+0.42$	+2.7
M-d90 vs U	3×6	$+0.78 \pm 0.24$	3.3 (7)	$+0.70$	+4.2
M-d90 vs M	3×6	$+0.06 \pm 0.23$	0.3 (7)	$+0.05$	-0.2

The agents share one MCTS recipe (200 simulations, 300,000 self-play episodes) and differ in their value function, the statistics table used during training, and the opponent model. Every arm uses three value-function seeds with one or two MCTS seeds on each: MAINTAINED, UNMAINTAINED, and STALE-2YR have 6 agents, and STALE-1YR, MAINTAINED-AT-CUTOFF, MAINTAINED-D90, and M-vol have 3. MAINTAINED-AT-CUTOFF uses MAINTAINED's value functions and trains its policy on statistics frozen at the cutoff; it is the maintained recipe trained at the cutoff and then left alone. STALE-1YR and STALE-2YR apply UNMAINTAINED's leak-free recipe at cutoffs one and two years earlier. M-vol is MAINTAINED trained on as many rows as STALE-2YR. Each agent's opponent model, which initializes its policy and plays against it in self-play, is trained only on games up to the agent's own cutoff (Section IX).

Each terminal draft is scored against the realized outcomes of later games: the average win rate of the heroes each side picked, the synergy of each side's pairs and its counters against the other side (pair and matchup win rates beyond what the heroes' own rates predict), and the rule-based degenerate-composition flag. The scoring games are the 292,902 ranked games of build 2.55.16.97039 and the four 2.55.17 builds (May through September 2026). No agent's statistics include them. The first 11,960 games of 2.55.16.97039 fell inside the pinned future window, on which we chose MAINTAINED-D90's statistics and compared opponent models; the 156,057 games of the 2.55.17 builds, which no part of any agent saw, are reported separately in Table IV. Inference uses crossed random effects for the two seed identities with a t reference and Satterthwaite degrees of freedom. The primary contrast is future-meta hero win rate; synergy, counters, degeneracy, and the 2.55.17 subset are secondary.

Staleness costs agents in proportion to its length. The

unmaintained agent trained at the 2026 cutoff leads the agent trained a year earlier by $+0.56 \pm 0.19$pp of future-meta hero win rate ($t = 2.9$, 8 df, $p = 0.02$) and the agent trained two years earlier by $+1.39 \pm 0.26$pp ($t = 5.4$, 10 df, $p < 0.001$; Table IV, Figure 2). The one-year agent in turn leads the two-year agent by $+0.70 \pm 0.26$pp ($t = 2.7$, 7 df, $p = 0.03$), so the two years are separate costs, and the two steps sum to 1.26pp, close to the direct two-year gap. By the realized-outcome index these gaps are worth 2.1, 2.5, and 4.7 points of team win probability. The maintained agent leads the one- and two-year agents by $+1.02$ and $+1.77$pp (4.7 and 7.7 points). Our corpus grows over time, so the two-year agent also trained on fewer games. A maintained agent trained on STALE-2YR's row count still leads it by $+1.71$pp, 97% of the full gap, so data volume explains almost none of it. Every staleness contrast keeps its sign and size on the 2.55.17 builds alone ($+0.66$ to $+1.82$pp).

Within one cutoff the effects are smaller. MAINTAINED leads UNMAINTAINED by $+0.58 \pm 0.28$pp ($t = 2.0$, 10 df, $p = 0.067$), about 3.2 points of team win probability. The lead is positive on both halves of the scoring set ($+0.67$pp on 2.55.16.97039, $+0.51$pp on the 2.55.17 builds) but not significant at six agents per side. The two agents differ twice: MAINTAINED's value function trained on statistics aligned with each game's era, and its policy trained on three more months of statistics. MAINTAINED-AT-CUTOFF separates the two, and neither step is significant at our seed counts: MAINTAINED leads MAINTAINED-AT-CUTOFF by $+0.21 \pm 0.25$pp ($p = 0.43$; 95% interval -0.37 to $+0.79$), and MAINTAINED-AT-CUTOFF leads UNMAINTAINED by $+0.39 \pm 0.28$pp ($p = 0.21$). On the accuracy side, three months of refresh are worth 0.03pp (Section IV). The decayed-statistics agent leads UNMAINTAINED by $+0.78 \pm 0.24$pp ($t = 3.3$, 7 df, $p = 0.014$) and is level with MAINTAINED ($+0.06 \pm 0.23$pp).

Pairwise metrics tell a different story. The stale agents' pairs are at least as good as the newer agents': the one-year agent's synergy exceeds UNMAINTAINED's by 0.35 ± 0.10 ($t = 3.6$, 9 df, $p = 0.005$), and no other staleness contrast shows a synergy difference ($|t| \leq 1.0$). Counters favor the two-year agent. Against STALE-2YR, every newer agent counters worse: STALE-1YR by 0.31 ± 0.09 ($p = 0.009$), M-vol by 0.24 ± 0.07 ($p = 0.005$), UNMAINTAINED by 0.24 ± 0.11 ($p = 0.052$), and MAINTAINED by 0.15 ± 0.09 ($p = 0.11$). Against STALE-1YR the newer agents counter slightly better, and neither gap is significant ($+0.25$, $p = 0.14$ and 0.25). One hypothesis, which we did not test, is that matchup advantages that held across builds favor an agent trained on two older years of data. Within the 2026 cutoff, the decayed-statistics agent's synergy exceeds UNMAINTAINED's by 0.35 ± 0.10 ($p = 0.005$), and MAINTAINED-AT-CUTOFF counters better than UNMAINTAINED by 0.51 ± 0.19 ($p = 0.03$). No matchup shows a significant difference in broken teams ($|t| \leq 1.2$); MAINTAINED and UNMAINTAINED draft them 26.4% and 27.0% of the time head-to-head, although the greedy protocol of Table II separates their value functions.

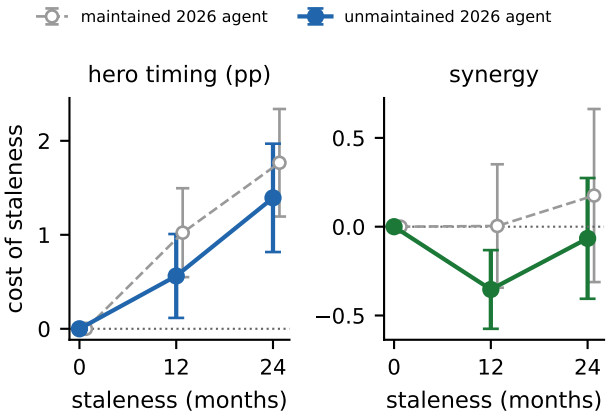


Fig. 2. Cost of staleness in future-meta hero win rate (left, pp) and synergy (right), scored on out-of-sample builds, with 95% t intervals from crossed seed random effects. Filled: the unmaintained agent trained at the 2026 cutoff against leak-free agents trained one and two years earlier (0 months is the reference). Hollow: the maintained 2026 agent against the same stale agents. Every agent's opponent model is trained to its own cutoff.

VI. What Drifts

Per-build statistics let us measure how each signal class decays. Correcting each correlation for binomial sampling noise (disattenuated r), role-composition win rates are nearly static, at 0.95 with themselves over a lag of 16 builds. Hero pick rates are nearly as stable (0.89 at lag 16). Per-hero win rate is the fastest-decaying reliable signal, falling from 0.94 at lag 1 to 0.62 at lag 16 (raw correlations 0.85 and 0.57). Raw pair win rates track their component hero win rates (0.61 at lag 16; Figure 3). The pairwise interaction beyond the two heroes' strengths is weak and mostly within-build sampling noise, a caution for pairwise features generally. The value model ages as well. Scored on the same rows, the seven builds after the 2026 cutoff, each with its own frozen statistics, the leak-free all-history model trained at the 2026 cutoff has calibration slope 0.96, and models trained one and two years earlier have 0.79 and 0.75 (three seeds each; seed ranges 0.93 to 0.99, 0.78 to 0.80, and 0.72 to 0.77).

Two questions a maintainer asks after every patch have answers here. First, how soon can a new build's own statistics be trusted? Measured against the build's remaining games, which is what a deployed system can check, patch-local hero win rates come within a point of the truth after a median of about 17,100 games into a build. (A popular alternative, distance to the build's final value, reaches the same threshold at 10,800 games, but the early games are part of the final value, and its error matches pure sampling error at every checkpoint, so it measures accumulation more than settling.) At the snapshot's early-2026 rate of about 11,000 ranked games per week that is about a week and a half, and at the 14,000 to 16,000 per week we measured for the pre-registration, about eight days. Second, does drift change which compositions never get played? The observed composition sets of consecutive builds have a mean Jaccard similarity of 0.81, and the per-build never-fielded count follows build volume ($r = -0.98$ against log games). Built from each

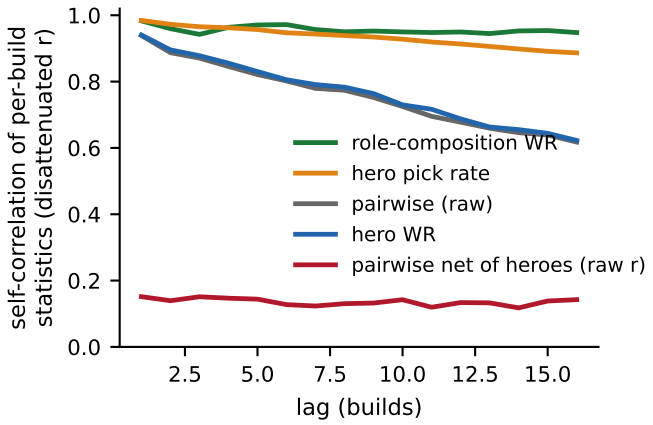


Fig. 3. Signal-class decay: self-correlation of per-build aggregate statistics against build lag, corrected for binomial sampling noise (disattenuated r), except the net pairwise series, which shows raw r because its reliability is near zero.

era's history, the never-fielded targets of our earlier paper's synthetic-augmentation method stay within 24 compositions per tier of the full-corpus target from about three years of history on, and within 2 at the cutoff; the earliest eras differ by up to 99.

VII. Detecting Balance Changes from Outcomes

A maintainer wants to know when the game changed. The developer's patch notes provide ground truth: we scripted 40 mapped patches, and five builds have no published notes and count as no-change builds. The detector tests each hero's win rate across each boundary between sizable builds (two-proportion test, at least 200 games per side) and keeps the shifts that survive Benjamini–Hochberg correction at $q = 0.05$ within the boundary. A boundary fires when at least one hero survives.

The ground truth has two variants, and we report both. The scripted truth counts every hero the notes mention, including heroes mentioned only for bug fixes; this is what the frozen detection code uses. A balance-only variant drops bug-fix-only mentions, which appear in 22 of the 40 mapped patches (458 mentions). At the hero level, 46 of the detector's 59 flags name a hero the notes mention (0.78), and 39 of 59 name a hero with a balance change (0.66). A randomly chosen hero at a random boundary is mentioned about 30% of the time. At the boundary level, the detector fires on 16 of the 21 boundaries whose notes mention a hero and on 1 of the 6 without (one-sided Fisher exact $p = 0.015$); under the balance-only truth, 15 of 19 against 2 of 8 ($p = 0.014$). Boundary precision, 16 of 17, sounds stronger than it is, because 21 of 27 boundaries carry mentions and random firing would already score 0.78 (binomial $p = 0.08$). The controls are few, and five of the six no-change boundaries fall inside one maintenance era (2.55.3). Two misses are roster-wide cleanup patches that mention 88 and 64 heroes; at 2.55.2.88122 all three flags fall outside the notes. Per-hero recall is low (0.06), because the notes list every individual change and most are too small to detect singly. The same test on pick and ban rates has hero-level recall 0.69 and precision 0.32; it picks up the community reacting, which spills across build boundaries and fires on hype as readily as on tuning.

How fast does it know? The first flagged hero at each firing boundary crosses an uncorrected $p < 0.05$ threshold a median of 2,500 games into the build. Over all 46 true-positive flags, the uncorrected first crossing has median 6,900 games. A Bonferroni-corrected sequential test never flags 7 of the 46 before the build ends, and flags the other 39 after a median of 18,800 games. The detector's fire decisions themselves use the full build.

As a refresh trigger the detector performs like a calendar. In the replay of Table III, refreshing a fixed model only when the detector fires gives +0.677pp regret, against +0.675 for 14 evenly spaced blind refreshes, +0.685 for a refresh every third build, and +0.662 for every build; the differences are within the replay's noise. The detector arm is a hindsight upper bound: which boundaries fire is decided with the full build and with the sizable-build list known in advance, and each refresh is then placed at the first-detection time of a hero the full-build test flags. Latency itself costs 0.003pp; the sparser schedule costs 0.012pp. Recomputing statistics is cheap, so saving refreshes has little value. Sweeping the false-discovery level across two orders of magnitude moves hero-level precision from 0.90 (17 of 19 flags) to 0.44, and every operating point lands within 0.05pp of every other on the replay. Of the replay's 36 builds, 20 are sizable; the detector fires on 14 of the 20 boundaries ending at them, one of which (2.55.10.94470) has no documented change.

The detector's value is as an alarm, and it needs little. It uses version identifiers, which almost every deployed system logs, and the magnitude of the outcome shift between versions; it needs no patch notes or labels. We also tested whether change timing could be recovered without version identifiers: a multi-scale changepoint detector over daily hero pick, ban, and win rates, tuned on pre-2025 boundaries (its best family was pick and ban rates), reaches held-out precision 0.18 at recall 0.27. With a $\pm$3-day tolerance, the 11 held-out boundaries cover about 77 of the 507 held-out days, so chance precision is about 0.15. Without version identifiers we could not reconstruct the change calendar.

VIII. Evaluators Drift Too

The obvious way to score the head-to-heads is the consensus of our win-probability models, the panel that adjudicated our earlier tournament. Those judges were trained on a snapshot that includes the future period, so they share the maintained agent's information advantage by construction, and their verdict is unidentified. Replacing them produced a finding about judges.

We trained naive-feature judges (hero identities, map, and tier; no aggregate statistics) in two corpus families and scored the 2,880 drafts of the MAINTAINED versus UNMAINTAINED head-to-head of Section V with every judge. Here the drafts are a test set; the question is how judges of different eras score the same drafts, against a realized-outcome verdict that favors

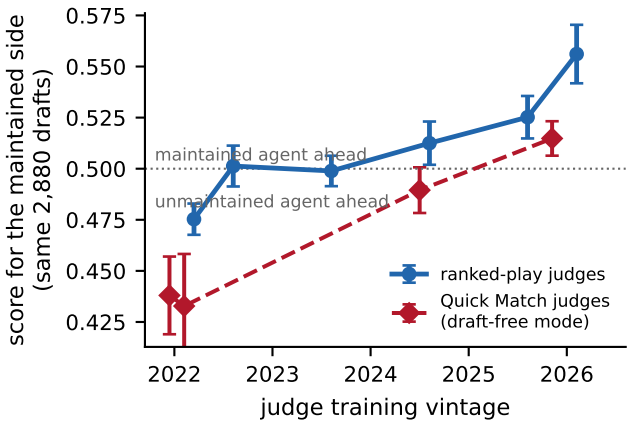


Fig. 4. The vintage matrix. Score assigned to the maintained side of the 2,880 MAINTAINED versus UNMAINTAINED drafts by win-probability judges trained at different times (95% intervals, normal approximation, clustered by seed pairing; judge-seed variation is not included). Realized outcomes favor the maintained side. Judges trained through early 2022 reverse the verdict, and judges from 2025 on agree with it. Diamonds: judges trained on Quick Match, a mode without a draft phase.

the maintained side. Six ranked-play judges span early 2022 through the cutoff, each trained on the 314,727 most recent games before its cutoff (158,701 for the earliest, 2022Q1). These judges' training data are subsets of both agents' training data, and the 2026 judge's window ends at the cutoff. Four Quick-Match judges (December 2021, early 2022, 2024, and July 2025 to the cutoff, on pools of 29K to 129K games) come from a mode with no draft phase: the matchmaker assembles teams, so these judges never saw a draft.

The ranked verdict rises with judge era: 0.475 (2022Q1), 0.501 (2022-07), 0.499 (2023), 0.513 (2024), 0.525 (2025), and 0.556 (2026). The Quick-Match family repeats the pattern: 0.438 (December 2021) and 0.433 (early 2022), then 0.490 (2024) and 0.515 (2026). Each score has two sources of error: draft sampling, which we estimate with standard errors clustered by seed pairing (0.004 to 0.013), and the judge's own training seed, whose standard deviation is 0.014 across three seeds of the 2026 ranked judge (range 0.027). Combining the two in quadrature, the judges trained through early 2022 declare the unmaintained agent the winner of a 2026 contest, by 1.7 to 3.7 combined standard errors; the judges from mid-2022 through 2024 are within one standard error of even; and the 2025 and 2026 judges side with the realized outcome, by 1.0 to 3.6. Because the Quick-Match family moves from about 0.43 to 0.52 with the game mode fixed, the reversal tracks the judge's era within one corpus family (Figure 4).

A judge carries the meta of its training window, and a contest decided by meta timing is where that matters most. Anyone evaluating agents in a drifting domain should date each judge and report the judge-by-era matrix, with the headline on realized outcomes.

IX. THE OPPONENT MODEL DRIFTS ON ITS OWN CLOCK

A deployed drafter also carries a behavioral model of its opponent (the next-pick predictor that guides search), and player behavior drifts on a different schedule than balance. We compared opponent models on 1.80 million next-pick samples from the seven future builds (Table V).

TABLE V
OPPONENT MODELS SCORED ON 1.80M FUTURE NEXT-PICK SAMPLES (TOP-1/TOP-5 ACCURACY, %), AVERAGED OVER TWO MODEL VARIANTS EXCEPT WHERE MARKED. ROWS 1 AND 4 SAW THE TEST DRAFTS IN TRAINING AND ARE REFERENCES ONLY. [a]RETRAINED BEFORE EACH TEST BUILD ON THE PRECEDING 30 DAYS OF PLAY. [b]ONE VARIANT.

Model (training data)	Top-1	Top-5
test-period fit (98% of the test drafts)	13.19	40.56
30 days before each build (rolling)[a]	**11.99**	–
30 days before the cutoff, trained once	11.84	**37.05**
full snapshot[b]	11.83	36.58
last 6 builds before the cutoff	11.73	37.01
all history to the cutoff	11.14	34.59
all history to 2.55.9.93613 (1-year-stale agents)[b]	10.54	32.41
all history to 2.55.4.91418 (2-year-stale agents)[b]	10.25	31.34

The model trained on the full snapshot, of which 7% postdates the cutoff, scores 0.75pp of top-1 above the all-history model deployable at the cutoff (same variant); that is a lower bound on behavioral drift. Recency recovers most of it: a model trained on the last six builds gains 0.59pp, and one trained once on the 30 days before the cutoff gains 0.70pp. The 30-day model's lead fades. It is the best deployable model on the three 2.55.15 builds (13.2%, 12.3%, and 12.2% top-1), and the worst on the last two, at 10.8% and 10.6% against 11.0% and 11.2% for the all-history model. Its gain is in bans (17.2% against 15.4%); on picks the two are level (8.6%). Rolled forward, retraining on the 30 days before each build, the window scores 11.99% overall, 0.15pp above the once-trained model and 0.85pp above the all-history model, and 11.4% on the last build instead of 10.6%. It still trails the all-history model on one build (2.55.16.96881, 10.6% against 11.0%), and the six-build window is steadier than either 30-day model, never below 11.2% on any build. Behavior favors recent play, retrained often.

Opponent models age with their agents too. Against the all-history model at the cutoff, which the 2026 agents train against (same variant), a model trained to 2.55.9.93613, one year before the cutoff, scores 0.54pp lower on the future builds (10.54%), and one trained to 2.55.4.91418, two years before, 0.84pp lower (10.25%). The order holds on each of the seven builds, and the first year costs more than the second.

Before running this comparison we set a threshold for retraining the search pipeline on refreshed opponents: 1.0pp of top-1 lost to behavioral drift. Neither the full-snapshot gap (0.75pp) nor the rolling window's gain (0.85pp) reaches it, so every agent in Section V searches against an opponent model trained to its own cutoff, on all history to that point.

X. THE OPERATING POLICY

The measurements above reduce to a short policy. Train on causal or out-of-fold statistics and check calibration on rows the statistics never contained. Retrain the value model on a fixed schedule; every three to six builds (quarterly

to twice yearly here) keeps regret within about 0.1pp of retraining constantly. Recompute the statistics at each retrain, and between retrains if it is free, which buys a few hundredths of a point. Prefer decayed statistics; our data do not pick a half-life between 90 and 365 days. Retrain the opponent model on recent play. Use the outcome-shift detector as an alarm that a build changed hero strength; do not use it to schedule refreshes.

Our production deployment retrains all three models monthly on the full corpus, with 90-day decayed statistics computed out of fold for training rows, and serves the same decayed statistics to the browser; a calibration-slope gate blocks deployment when the value model's slope on held-out rows leaves [0.8, 1.25]. The first full run of this pipeline completed and deployed on September 30, 2026 (slope 1.05); scheduled monthly runs begin November 1, 2026. Builds have averaged about a month apart, so a monthly retrain is close to retraining after every build, more often than the replay says is necessary.[2]

XI. A Prospective Test, Pre-Registered

Every result above is retrospective: the cutoff was chosen, then the future window was scored. So before submission we registered three forward claims on OSF (September 27, 2026; https://osf.io/3cxvd/overview?view_only=dbe817f56afe4507865e44bf24510748), with the analysis code frozen by commit hash: (1) the outcome-shift detector, decision rule untouched, fires on new builds whose patch notes contain hero changes and stays silent on no-change builds (builds whose changes are all individually minor are reported descriptively and adjudicate nothing); (2) the existing 4,500-draft head-to-head between MAINTAINED and UNMAINTAINED-LEAKY, rescored with the out-of-sample protocol of Section V against the new builds' realized statistics, keeps a positive maintained advantage in future-meta hero win rate (both agents and all drafts predate those builds, so nothing in this claim can be tuned). The registered rule for C2 is a normal-reference test of the crossed-random-effects estimate: confirmed if $z \geq 1.645$ (one-sided $\alpha = 0.05$), falsified if the estimate is at most 0, otherwise inconclusive. (3) Every production refresh event is logged and reported against the detector's fires. The claims cover only builds that ship after the registration date. The builds between our snapshot and the registration (2.55.16.97039 and the four 2.55.17 builds) already serve as the scoring set of Section V and are excluded. The analysis date is fixed at submission: to be stated here and on OSF at submission. New ranked games arrived at 14,000 to 16,000 per week when we registered, so a new build reaches the 20,000-game detector gate in under two weeks, and a six-month window should contain three or four sizable builds. The first logged production refresh under C3 is the September 30, 2026 deployment (Section X).

Three disclosures bear on the registered claims and will be repeated when they are reported. The registration text says bug-fix-only and ARAM-only hero mentions are excluded from the ground truth, as in the retrospective analysis. The frozen code does not exclude them: it counts every hero the notes mention. The registered test is the frozen code, so C1 will be adjudicated with the any-mention truth, and we will also report the balance-only variant. The retrospective numbers in Section VII report both. Second, the registered head-to-head (C2) was drafted by UNMAINTAINED-LEAKY, the unmaintained agent trained on leaky statistics, which was the agent available at registration. We will report it as registered and report the leak-free matchups of Section V alongside it as descriptive results. Third, on September 30, 2026 our database relabeled skill tiers into a new scheme (Section II), while the registered drafts carry the snapshot's labels. The frozen truth-scoring code groups games by the database label, so a rerun would score drafts against statistics from a different tier grouping. We will reconstruct the snapshot-scheme label from each game's stored league tier before scoring, and report this as a deviation.

[2]https://draft-assistant-review.vercel.app

XII. Related Work

Concept drift and dataset shift have a large literature, most of it classifier-centric: detect distributional change, adapt the model, measure accuracy [2], [3], [4]. The canonical detectors monitor a supervised error stream (DDM tracks error-rate inflation [6]; ADWIN maintains adaptive windows over it [7]), and adaptation means retraining or reweighting the classifier. Change-point detection supplies the standard machinery [5]; our detector is deliberately plain, and what we add is the ground truth we score it against (developer patch notes) and its price as a maintenance trigger. Our results sit on the retraining side of that literature: in this domain, retraining on a schedule does almost all of the repair, and the policy-level cost of drift is larger than the accuracy cost suggests. Target leakage through aggregate statistics is a known failure in tabular learning [8], and ordered or out-of-fold target statistics are the standard remedy [9]; we show how it distorts a drift study specifically, by making the unmaintained baseline look worse than drift does.

Continual and non-stationary reinforcement learning study policies that keep learning as the environment changes [10], [11], with catastrophic forgetting as a central obstacle [12]. The deployed case we study fixes the policy between maintenance events and asks when and what to touch; a continual learner would still need a trigger and leak-free statistics. OpenAI Five kept one Dota 2 agent current across game patches by "surgery" on its network and continued training [13], a retraining-side answer to the same problem at far larger scale. On the operations side, data dependencies and feature freshness are a known maintenance surface of deployed ML [14], production-readiness rubrics score monitoring loops [15], data-validation systems treat stale statistics as a failure [16], and practitioners report retraining cadence as an open question [17]; we supply a measured decomposition of what refresh and retraining each buy in one domain.

In games, drafting has been treated as prediction and recommendation, from pick prediction in Dota 2 [18] to team-aware hero recommendation [19] and personalized draft recommendation [20]. In this journal, Chen et al. learn to

draft with neural networks and MCTS [21], and Gourdeau and Archambault select heroes in professional Heroes of the Storm and Dota 2 drafts with a discriminative network [22]. Balance evaluation has its own methods [23]. To our knowledge none of this work measures maintenance policies against patch-note ground truth or prices staleness in head-to-head play. Offline reinforcement learning under non-stationarity is usually posed as a training-time problem [24]; here the policy is fixed and the world moves. This paper is the time-axis companion to our draft-policy paper [1]. Player-skill drift is out of scope and is the subject of a companion study of personalization.

XIII. Limitations

Everything here is one game and one major patch, a scoping that buys 45 builds with a fixed hero pool at the cost of external validity. The deployment replay uses one training seed per cell; orderings among regrets from different models within about 0.07pp (for example 0.050 and 0.067) are at noise level, although paired comparisons on the same models, such as refresh against no refresh at a fixed cadence, are not. The judge-free protocol rests on realized hero, pair, and matchup statistics, which are marginal quantities; we mitigate by triangulating the metrics and by combining them with weights fitted on held-out real games. Agent-level comparisons rest on three to six agents per arm, one or two per value-function seed. That budget resolves the staleness contrasts but not the same-cutoff ones: the maintained agent's lead over the unmaintained agent has $p = 0.067$, and its split between the value-function recipe and three months of refresh is not resolved. The one-year arm was cut from 6 agents to 3 when compute was capped, so the staleness gradient's one-year point rests on three agents. We tested about twenty accuracy cells on the same future window and many matchup-by-metric combinations; we name the primary contrasts in each section and treat the rest as secondary.

XIV. Conclusion

A draft policy that nobody maintains loses about a point of accuracy over three years of balance changes. Retraining on a schedule of three to six builds recovers nearly all of it; refreshing statistics with the weights fixed recovers a third, and adds a few hundredths of a point once retraining happens. At the agent level, one and two years without retraining cost 0.6 and 1.4 points of future-meta hero win rate against an agent trained at the cutoff, while three months of refresh and the same-cutoff recipe gap are too small to resolve with our seed budget. Accuracy understates the policy-level damage, and learned judges are as era-bound as the models they score.

AI Disclosure

Portions of this manuscript's text were drafted and edited with Claude (Anthropic) [25]; the authors designed and verified all experiments and approved the text.

References

[1] Anonymous authors, "Diagnosing and repairing out-of-distribution failure in MOBA draft policies," under review, 2026.

[2] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. MIT Press, 2009.

[3] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, 2014.

[4] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[5] S. Aminikhanghahi and D. J. Cook, "A survey of methods for time series change point detection," *Knowledge and Information Systems*, vol. 51, no. 2, 2017.

[6] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence (SBIA)*, 2004.

[7] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM International Conference on Data Mining*, 2007.

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, 2012.

[9] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.

[10] S. Padakandla, "A survey of reinforcement learning algorithms for dynamically varying environments," *ACM Computing Surveys*, vol. 54, no. 6, 2021.

[11] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1401–1476, 2022.

[12] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[13] C. Berner *et al.*, "Dota 2 with large scale deep reinforcement learning," arXiv:1912.06680, 2019.

[14] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, 2015.

[15] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *IEEE International Conference on Big Data*, 2017.

[16] N. Polyzotis, M. Zinkevich, S. Roy, E. Breck, and S. Whang, "Data validation for machine learning," in *Proc. Machine Learning and Systems (MLSys)*, 2019.

[17] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, "Operationalizing machine learning: An interview study," arXiv:2209.09125, 2022.

[18] A. Summerville, M. Cook, and B. Steenhuisen, "Draft-analysis of the Ancients: Predicting draft picks in DotA 2 using machine learning," in *AIIDE Workshop on Experimental AI in Games*, 2016.

[19] Z. Chen, T.-H. D. Nguyen, Y. Xu, C. Amato, S. Cooper, Y. Sun, and M. S. El-Nasr, "The art of drafting: A team-oriented hero recommendation system for multiplayer online battle arena games," in *Proc. ACM Conference on Recommender Systems (RecSys)*, 2018.

[20] H. Lee, D. Hwang, H. Kim, B. Lee, and J. Choo, "DraftRec: Personalized draft recommendation for winning in multi-player online battle arena games," in *Proc. ACM Web Conference (WWW)*, 2022.

[21] S. Chen, M. Zhu, D. Ye, W. Zhang, Q. Fu, and W. Yang, "Which heroes to pick? Learning to draft in MOBA games with neural networks and tree search," *IEEE Transactions on Games*, vol. 13, no. 4, pp. 410–421, 2021.

[22] D. Gourdeau and L. Archambault, "Discriminative neural network for hero selection in professional Heroes of the Storm and DOTA 2," *IEEE Transactions on Games*, vol. 13, no. 4, pp. 380–387, 2021.

[23] A. Jaffe, A. Miller, E. Andersen, Y.-E. Liu, A. Karlin, and Z. Popović, "Evaluating competitive game balance with restricted play," in *Proc. AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2012.

[24] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," arXiv:2005.01643, 2020.

[25] Anthropic, "Claude (Fable 5)," large language model, 2026.